

UNIVERSIDADE SÃO FRANCISCO
Análise e Desenvolvimento de Sistemas
Engenharia de Software
EV 50775 – Prática Profissional: Projeto de Software

**GYMCONTROL:
SISTEMA DE GERENCIAMENTO DE ACADEMIA**

Planejamento do Sistema

Itatiba
Junho de 2026

GYMCONTROL: SISTEMA DE GERENCIAMENTO DE ACADEMIA

Trabalho apresentado à disciplina EV 50775 – Prática Profissional: Projeto de Software do Curso de Análise e Desenvolvimento de Sistemas da Universidade São Francisco, sob orientação do(a) Prof.(a) Leandro Felipe Carvalho, como requisito parcial para obtenção de média semestral.

Itatiba
Junho de 2026

SUMÁRIO

1 INTRODUÇÃO.....	7
2 DESCRITIVO COMPUTACIONAL DO SOFTWARE.....	8
3 CARACTERIZAÇÃO DO SISTEMA.....	9
3.1 Nome do sistema	9
3.2 Objetivo do sistema.....	9
3.3 Público-alvo.....	9
3.4 Problema identificado.....	9
3.5 Solução proposta	9
4 REQUISITOS DO SISTEMA.....	10
4.1 Requisitos funcionais.....	10
4.2 Requisitos não funcionais.....	10
5 MODELAGEM DO SISTEMA (DIAGRAMAS UML).....	12
5.1 Diagramas comportamentais da UML	12
5.1.1 Diagrama de Caso de Uso.....	12
5.1.2 Descrição dos casos de uso	14
5.1.3 Diagrama de Sequência.....	15
5.2 Diagramas estruturais da UML	17
5.2.1 Diagrama de Classe	17
5.2.2 Diagrama de Componentes	19
6 TECNOLOGIAS UTILIZADAS	21
7 CRONOGRAMA DE DESENVOLVIMENTO	22
8 ORÇAMENTO PARA DESENVOLVIMENTO.....	23
9 IMPLEMENTAÇÃO REALIZADA	24
9.1 Visão geral	24
9.2 Arquitetura.....	24
9.3 Backend	24
9.4 Frontend.....	25
9.5 Banco de dados	25
9.6 Endurecimento de segurança (RNF02 e RNF07)	25
9.7 Testes automatizados	26
9.8 Containerização	26
9.9 Integração contínua e entrega contínua (CI/CD)	26
9.10 Deploy.....	27
9.11 Rastreabilidade de requisitos	27

Requisitos funcionais	27
Requisitos não funcionais	28
10 ACESSO E EXECUÇÃO DO SOFTWARE	29
10.1 Frontend no ar	29
10.2 Repositório de código	29
10.3 Execução local com Docker	30
10.4 Documentação adicional	30
11 CONSIDERAÇÕES FINAIS	30
REFERÊNCIAS	32

LISTA DE QUADROS

QUADRO 1 – Requisitos funcionais do sistema GymControl.....	10
QUADRO 2 – Requisitos não funcionais do sistema GymControl.....	10
QUADRO 3 – Atores e casos de uso do sistema GymControl	13
QUADRO 4 – Descrição do caso de uso Cadastrar Aluno	14
QUADRO 5 – Descrição do caso de uso Registrar Pagamento	14
QUADRO 6 – Descrição do caso de uso Cadastrar Treino	14
QUADRO 7 – Sequência de mensagens — registrar pagamento de mensalidade	16
QUADRO 8 – Principais classes do sistema e seus atributos	18
QUADRO 9 – Relacionamentos entre as classes do sistema	18
QUADRO 10 – Componentes do sistema GymControl	20
QUADRO 11 – Tecnologias escolhidas para o desenvolvimento do sistema.....	21
QUADRO 12 – Cronograma de desenvolvimento do sistema	22
QUADRO 13 – Estimativa de orçamento para o desenvolvimento.....	23

LISTA DE FIGURAS

FIGURA 1 – Diagrama de Caso de Uso do sistema GymControl.....	13
FIGURA 2 – Diagrama de Sequência: registrar pagamento de mensalidade	16
FIGURA 3 – Diagrama de Classe do sistema GymControl	18
FIGURA 4 – Diagrama de Componentes do sistema GymControl	20

1 INTRODUÇÃO

O presente trabalho tem como objetivo apresentar o planejamento de um sistema de gerenciamento de academia, denominado GymControl. O sistema foi proposto com a finalidade de informatizar processos administrativos e operacionais comuns em academias, como cadastro de alunos, controle de planos, registro de pagamentos, acompanhamento de treinos e controle de frequência.

A escolha desse tema se justifica pela necessidade de organização das informações em academias de pequeno e médio porte, que muitas vezes ainda utilizam métodos manuais ou planilhas para controlar seus dados. Conforme destaca Pressman (2016), a engenharia de software fornece as bases metodológicas necessárias para o desenvolvimento de soluções que substituam processos manuais por sistemas automatizados, reduzindo falhas, agilizando atendimentos e melhorando a gestão de informações.

O problema que motiva este trabalho pode ser sintetizado na seguinte questão: como informatizar, de forma simples e acessível, os processos de gestão de uma academia de pequeno ou médio porte, garantindo organização, controle e disponibilidade das informações? A hipótese adotada é que um sistema web, desenvolvido com tecnologias amplamente difundidas, é capaz de centralizar as informações de alunos, professores, planos, pagamentos, treinos e frequência, oferecendo uma alternativa viável ao controle manual atualmente praticado.

O objetivo geral deste trabalho é planejar o desenvolvimento de um sistema web de gerenciamento de academia. Como objetivos específicos, propõe-se: (i) descrever computacionalmente o software; (ii) caracterizar o sistema, definindo seus usuários e funcionalidades; (iii) levantar os requisitos funcionais e não funcionais; (iv) modelar o sistema por meio de diagramas UML, conforme orientações de Booch, Rumbaugh e Jacobson (2006); (v) definir as tecnologias para o desenvolvimento; e (vi) estimar o cronograma e o orçamento do projeto.

O trabalho está organizado em onze capítulos. Após esta introdução, o capítulo 2 apresenta o descritivo computacional do software. O capítulo 3 caracteriza o sistema. O capítulo 4 lista os requisitos funcionais e não funcionais. O capítulo 5 apresenta os diagramas UML que modelam o sistema. O capítulo 6 descreve as tecnologias escolhidas. O capítulo 7 traz o cronograma de desenvolvimento. O capítulo 8 apresenta o orçamento estimado. O capítulo 9 detalha a implementação efetivamente realizada. O capítulo 10 informa como acessar e executar o software. Por fim, o capítulo 11 traz as considerações finais. As referências utilizadas estão arroladas ao final do trabalho.

2 DESCRITIVO COMPUTACIONAL DO SOFTWARE

O GymControl é um sistema desenvolvido para auxiliar no gerenciamento de uma academia, permitindo o controle de alunos, planos, pagamentos, treinos, professores e frequência dos alunos.

Atualmente, muitas academias pequenas ainda realizam parte do controle de forma manual, usando planilhas, cadernos ou mensagens avulsas. Isso pode gerar problemas como perda de informações, dificuldade no controle de mensalidades, falta de organização dos treinos e ausência de histórico dos alunos.

O sistema proposto tem como objetivo informatizar esse processo, oferecendo uma plataforma simples onde os funcionários da academia possam cadastrar alunos, registrar planos, controlar pagamentos, montar treinos e consultar informações de forma rápida.

O sistema terá três tipos principais de usuários. O Administrador é responsável por gerenciar alunos, professores, planos, pagamentos e relatórios. O Professor é responsável por cadastrar e acompanhar os treinos dos alunos. O Aluno poderá consultar seu plano, seus treinos e sua situação de pagamento.

As principais funcionalidades do sistema serão: cadastro de alunos; cadastro de professores; cadastro de planos da academia; controle de mensalidades; registro de pagamentos; cadastro de treinos; consulta de frequência; consulta de alunos ativos e inadimplentes; e emissão de relatórios simples.

3 CARACTERIZAÇÃO DO SISTEMA

3.1 Nome do sistema

O sistema recebe a denominação GymControl – Sistema de Gerenciamento de Academia.

3.2 Objetivo do sistema

O objetivo do sistema é facilitar o controle administrativo e operacional de uma academia, organizando informações sobre alunos, professores, planos, mensalidades, treinos e frequência.

3.3 Público-alvo

O sistema é voltado para academias de pequeno e médio porte que desejam informatizar seus processos internos.

3.4 Problema identificado

Academias que utilizam métodos manuais para controlar alunos, pagamentos e treinos podem enfrentar dificuldades como: dados desorganizados; dificuldade para identificar alunos inadimplentes; falta de histórico dos treinos; controle manual de frequência; atraso no atendimento; e risco de perda de informações.

3.5 Solução proposta

A solução proposta é um sistema web simples que permita centralizar as principais informações da academia, facilitando o acesso, o controle e a organização dos dados. Sommerville (2019) argumenta que sistemas web bem projetados oferecem flexibilidade de acesso e baixo custo de implantação, características compatíveis com a realidade de academias de pequeno e médio porte.

4 REQUISITOS DO SISTEMA

Esta seção apresenta os requisitos funcionais e não funcionais do sistema GymControl. Conforme Sommerville (2019), os requisitos funcionais descrevem o que o sistema deve fazer, enquanto os requisitos não funcionais expressam restrições, propriedades de qualidade e características de operação.

4.1 Requisitos funcionais

O QUADRO 1 apresenta a lista de requisitos funcionais identificados para o sistema.

QUADRO 1 – Requisitos funcionais do sistema GymControl

Código	Requisito funcional
RF01	O sistema deve permitir o cadastro de alunos.
RF02	O sistema deve permitir a edição e exclusão de alunos.
RF03	O sistema deve permitir o cadastro de professores.
RF04	O sistema deve permitir o cadastro de planos da academia.
RF05	O sistema deve permitir registrar pagamentos de mensalidades.
RF06	O sistema deve permitir consultar alunos inadimplentes.
RF07	O sistema deve permitir o cadastro de treinos.
RF08	O sistema deve permitir vincular treinos aos alunos.
RF09	O sistema deve permitir registrar a frequência dos alunos.
RF10	O sistema deve gerar relatórios simples de alunos, pagamentos e frequência.

Fonte: elaborado pelos autores.

4.2 Requisitos não funcionais

O QUADRO 2 apresenta os requisitos não funcionais, que estabelecem as características de qualidade e restrições do sistema.

QUADRO 2 – Requisitos não funcionais do sistema GymControl

Código	Requisito não funcional
RNF01	O sistema deve possuir interface simples e intuitiva.
RNF02	O sistema deve ter controle de acesso por tipo de usuário.
RNF03	O sistema deve armazenar os dados em banco de dados relacional.
RNF04	O sistema deve ser acessível por navegador web.

Código	Requisito não funcional
RNF05	O sistema deve ter boa organização visual.
RNF06	O sistema deve permitir consultas rápidas.
RNF07	O sistema deve proteger informações básicas dos usuários.

Fonte: elaborado pelos autores.

5 MODELAGEM DO SISTEMA (DIAGRAMAS UML)

A Unified Modeling Language (UML) é uma linguagem-padrão para modelagem de sistemas orientados a objetos, amplamente utilizada na indústria de software (BOOCH; RUMBAUGH; JACOBSON, 2006). A UML organiza seus diagramas em duas grandes categorias: os diagramas comportamentais, que representam o comportamento dinâmico do sistema, e os diagramas estruturais, que representam sua estrutura estática. Esta seção apresenta os diagramas das duas categorias utilizados para documentar o sistema GymControl, acompanhados de quadros complementares que descrevem atores, casos de uso, classes, relacionamentos, mensagens e componentes.

5.1 Diagramas comportamentais da UML

Os diagramas comportamentais representam o comportamento dinâmico do sistema, descrevendo como os objetos interagem e como o sistema responde aos eventos. Para o GymControl, foram elaborados o Diagrama de Caso de Uso e o Diagrama de Sequência, apresentados a seguir.

5.1.1 Diagrama de Caso de Uso

O diagrama de caso de uso identifica os atores do sistema e as funcionalidades disponíveis para cada tipo de usuário. A FIGURA 1 apresenta o diagrama UML completo, e o QUADRO 3 sintetiza os três atores do sistema GymControl e seus respectivos casos de uso.

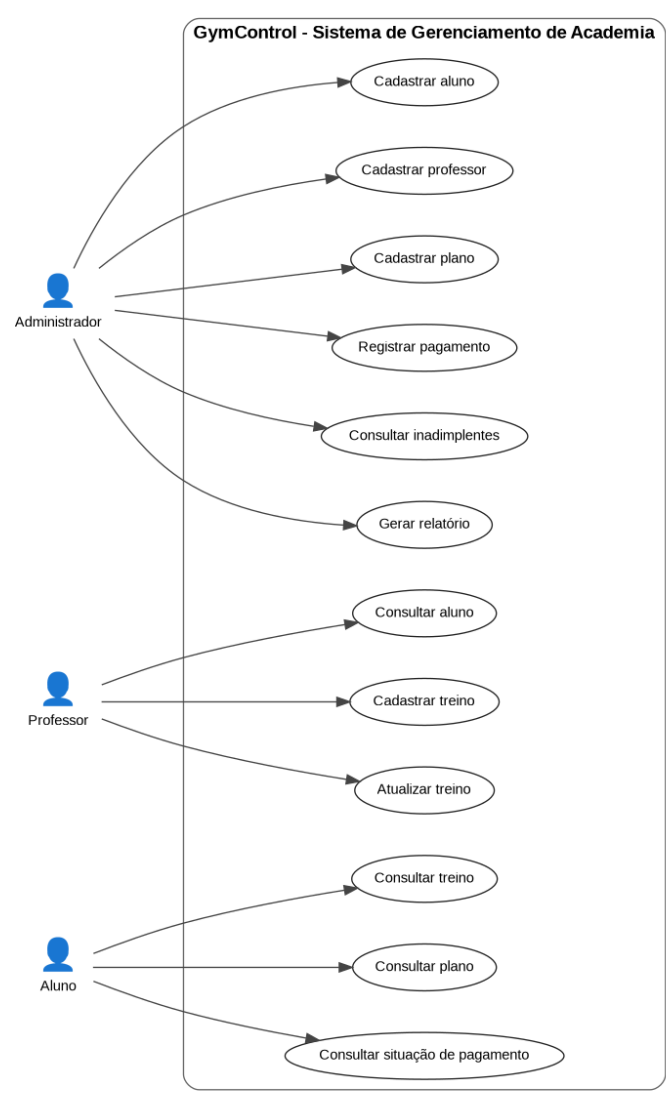


FIGURA 1 – Diagrama de Caso de Uso do sistema GymControl

Fonte: elaborado pelos autores.

QUADRO 3 – Atores e casos de uso do sistema GymControl

Ator	Casos de uso
Administrador	Cadastrar aluno; cadastrar professor; cadastrar plano; registrar pagamento; consultar inadimplentes; gerar relatório.
Professor	Consultar aluno; cadastrar treino; atualizar treino.
Aluno	Consultar treino; consultar plano; consultar situação de pagamento.

Fonte: elaborado pelos autores.

5.1.2 Descrição dos casos de uso

Os quadros a seguir (QUADRO 4, QUADRO 5 e QUADRO 6) detalham, respectivamente, os casos de uso Cadastrar Aluno, Registrar Pagamento e Cadastrar Treino, descrevendo ator principal, objetivo, pré-condição, fluxo principal e pós-condição de cada um.

QUADRO 4 – Descrição do caso de uso Cadastrar Aluno

Item	Cadastrar Aluno
Ator principal	Administrador
Objetivo	Registrar um novo aluno no sistema.
Pré-condição	O administrador deve estar logado.
Fluxo principal	O administrador acessa a tela de cadastro, informa os dados do aluno, escolhe o plano e salva o cadastro.
Pós-condição	O aluno fica registrado no sistema.

Fonte: elaborado pelos autores.

QUADRO 5 – Descrição do caso de uso Registrar Pagamento

Item	Registrar Pagamento
Ator principal	Administrador
Objetivo	Registrar o pagamento da mensalidade de um aluno.
Pré-condição	O aluno deve estar cadastrado.
Fluxo principal	O administrador pesquisa o aluno, informa o valor pago, a data de pagamento e confirma o registro.
Pós-condição	O pagamento fica registrado no sistema.

Fonte: elaborado pelos autores.

QUADRO 6 – Descrição do caso de uso Cadastrar Treino

Item	Cadastrar Treino
Ator principal	Professor
Objetivo	Criar um treino personalizado para o aluno.
Pré-condição	O professor e o aluno devem estar cadastrados.
Fluxo principal	O professor seleciona o aluno, informa os exercícios, séries, repetições e observações.
Pós-condição	O treino fica vinculado ao aluno.

Fonte: elaborado pelos autores.

5.1.3 Diagrama de Sequência

O cenário escolhido para o diagrama de sequência foi o registro de pagamento de mensalidade. A FIGURA 2 apresenta o diagrama de sequência UML, e o QUADRO 7 complementa com a sequência detalhada de mensagens trocadas entre os participantes (Administrador, Sistema e Banco de Dados). As mensagens estão numeradas em ordem cronológica de envio.

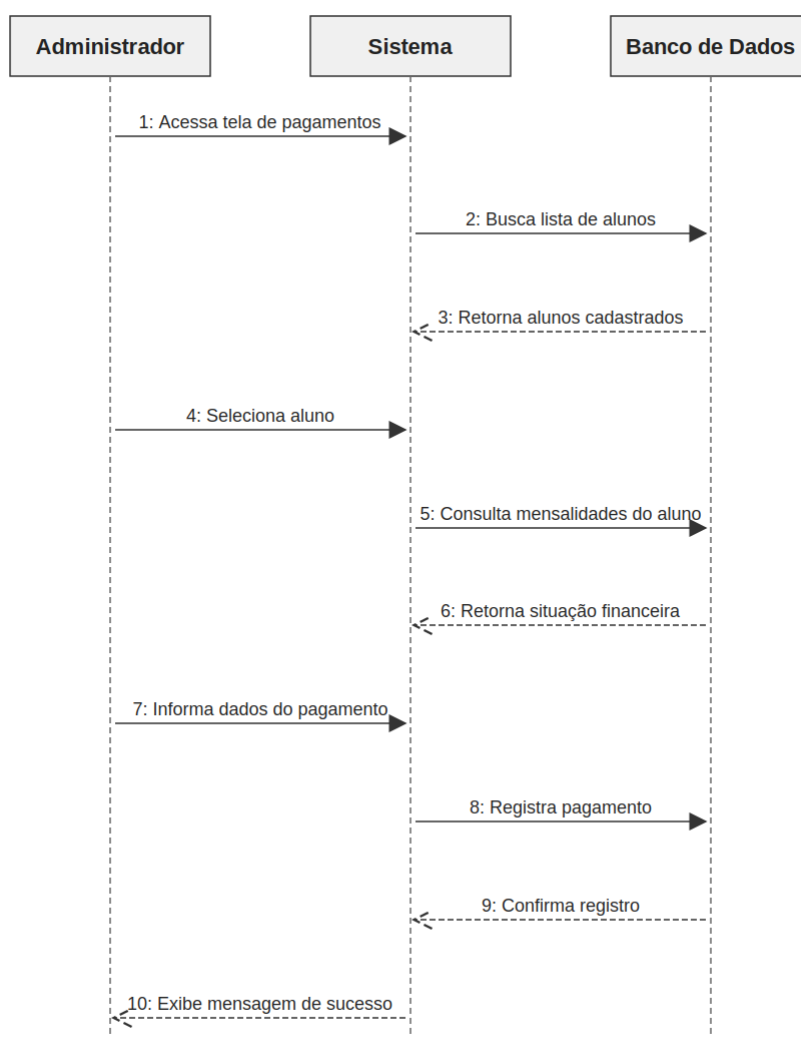


FIGURA 2 – Diagrama de Sequência: registrar pagamento de mensalidade

Fonte: elaborado pelos autores.

QUADRO 7 – Sequência de mensagens — registrar pagamento de mensalidade

Nº	De	Para	Mensagem
1	Administrador	Sistema	Acessa a tela de pagamentos.
2	Sistema	Banco de Dados	Busca a lista de alunos cadastrados.
3	Banco de Dados	Sistema	Retorna os alunos cadastrados.
4	Administrador	Sistema	Seleciona o aluno desejado.
5	Sistema	Banco de Dados	Consulta as mensalidades do aluno.
6	Banco de Dados	Sistema	Retorna a situação financeira do aluno.
7	Administrador	Sistema	Informa os dados do pagamento.
8	Sistema	Banco de Dados	Registra o pagamento.
9	Banco de Dados	Sistema	Confirma o registro.

Nº	De	Para	Mensagem
10	Sistema	Administrador	Exibe mensagem de sucesso.

Fonte: elaborado pelos autores.

5.2 Diagramas estruturais da UML

Os diagramas estruturais representam a estrutura estática do sistema, ou seja, as classes, componentes e demais elementos que compõem sua arquitetura. Para o GymControl, foram elaborados o Diagrama de Classe e o Diagrama de Componentes, apresentados a seguir.

5.2.1 Diagrama de Classe

O diagrama de classes apresenta as principais classes do sistema, seus atributos e os relacionamentos entre elas. A FIGURA 3 apresenta o diagrama de classes UML, o QUADRO 8 sintetiza as classes e seus principais atributos e o QUADRO 9 descreve os relacionamentos entre as classes.

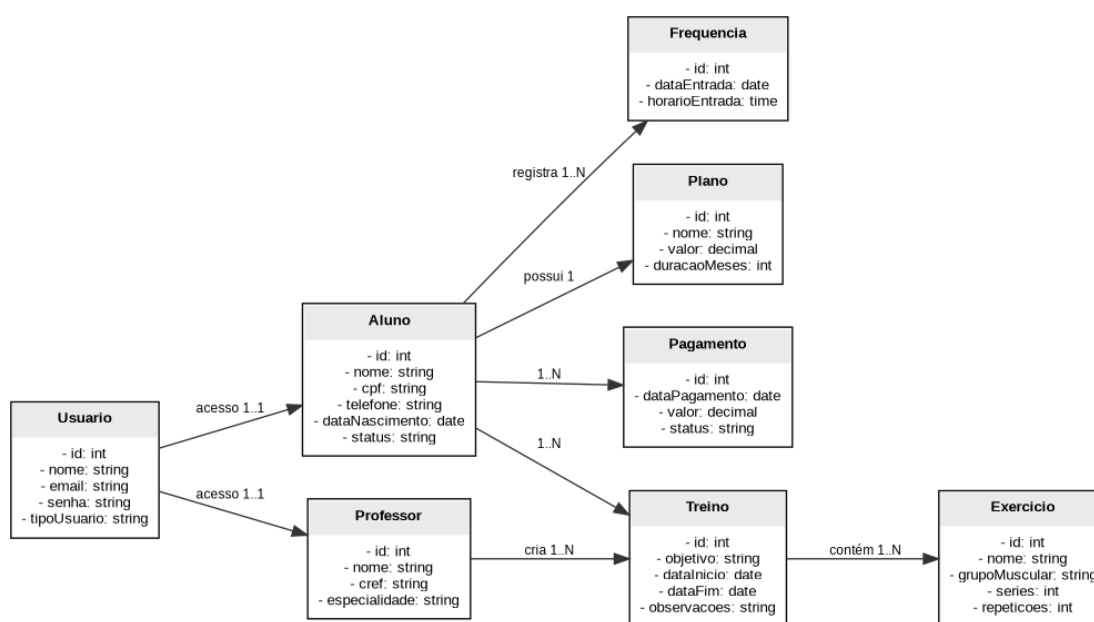


FIGURA 3 – Diagrama de Classe do sistema GymControl

Fonte: elaborado pelos autores.

QUADRO 8 – Principais classes do sistema e seus atributos

Classe	Principais atributos
Usuario	id, nome, email, senha, tipoUsuario
Aluno	id, nome, cpf, telefone, dataNascimento, status
Professor	id, nome, cref, especialidade
Plano	id, nome, valor, duracaoMeses
Pagamento	id, dataPagamento, valor, status
Treino	id, objetivo, dataInicio, dataFim, observacoes
Exercicio	id, nome, grupoMuscular, series, repeticoes
Frequencia	id, dataEntrada, horarioEntrada

Fonte: elaborado pelos autores.

QUADRO 9 – Relacionamentos entre as classes do sistema

Classes envolvidas	Cardinalidade	Descrição
Usuario – Aluno	1..1	Um usuário corresponde a um aluno (acesso de aluno ao sistema).
Usuario – Professor	1..1	Um usuário corresponde a um professor (acesso de professor ao sistema).
Aluno – Plano	N..1	Um aluno possui um plano.
Aluno – Pagamento	1..N	Um aluno pode possuir vários pagamentos.
Aluno – Treino	1..N	Um aluno pode possuir um ou mais treinos.

Classes envolvidas	Cardinalidade	Descrição
Aluno – Frequencia	1..N	Um aluno possui vários registros de frequência.
Professor – Treino	1..N	Um professor pode criar vários treinos.
Treino – Exercício	1..N	Um treino contém vários exercícios.

Fonte: elaborado pelos autores.

5.2.2 Diagrama de Componentes

O diagrama de componentes mostra a divisão do sistema em interface web, back-end e banco de dados. A FIGURA 4 apresenta o diagrama de componentes UML e o QUADRO 10 relaciona cada tela do sistema (interface web) ao componente de controle correspondente no back-end, indicando que todos os controles acessam o mesmo banco de dados relacional MySQL.

6 TECNOLOGIAS UTILIZADAS

Para o desenvolvimento do sistema, foram escolhidas tecnologias simples, acessíveis e adequadas para um projeto acadêmico. O QUADRO 11 apresenta a relação das tecnologias selecionadas, suas finalidades e respectivas justificativas. As ferramentas Lucidchart, Visual Paradigm e Astah UML foram utilizadas para apoiar a etapa de modelagem dos diagramas UML.

QUADRO 11 – Tecnologias escolhidas para o desenvolvimento do sistema

Tecnologia	Finalidade	Justificativa
JavaScript	Linguagem de programação	Linguagem popular, versátil e adequada para front-end e back-end.
HTML e CSS	Estrutura e estilo das telas	Tecnologias básicas para construção de interfaces web.
Node.js	Back-end	Permite desenvolver a lógica do sistema usando JavaScript no servidor.
MySQL	Banco de dados	Banco relacional adequado para armazenar alunos, planos, pagamentos e treinos.
Visual Studio Code	Editor de código	Ferramenta gratuita e amplamente utilizada no desenvolvimento web.
Lucidchart, Visual Paradigm ou Astah UML	Modelagem UML	Ferramentas apropriadas para a criação dos diagramas solicitados.

Fonte: elaborado pelos autores.

7 CRONOGRAMA DE DESENVOLVIMENTO

O QUADRO 12 apresenta o cronograma estimado para o desenvolvimento do sistema, organizado em treze etapas sequenciais que abrangem desde o levantamento de requisitos até a entrega final.

QUADRO 12 – Cronograma de desenvolvimento do sistema

Etapas	Atividade	Prazo estimado
1	Definição do tema e levantamento dos requisitos	2 dias
2	Descrição das funcionalidades do sistema	2 dias
3	Criação do Diagrama de Caso de Uso	2 dias
4	Criação do Diagrama de Sequência	2 dias
5	Criação do Diagrama de Classes	3 dias
6	Criação do Diagrama de Componentes	2 dias
7	Definição das tecnologias utilizadas	1 dia
8	Criação do banco de dados	3 dias
9	Desenvolvimento das telas do sistema	5 dias
10	Desenvolvimento das funcionalidades principais	7 dias
11	Testes e correções	4 dias
12	Revisão do relatório final	2 dias
13	Entrega final do sistema e documentação	1 dia

Fonte: elaborado pelos autores.

8 ORÇAMENTO PARA DESENVOLVIMENTO

O orçamento apresentado no QUADRO 13 é uma estimativa acadêmica para o desenvolvimento do sistema, considerando os profissionais envolvidos, a quantidade de horas estimadas para cada atividade e o valor por hora praticado no mercado.

QUADRO 13 – Estimativa de orçamento para o desenvolvimento

Profissional	Atividade	Horas	Valor/hora	Total
Analista de Sistemas	Levantamento de requisitos e documentação	20h	R\$ 60,00	R\$ 1.200,00
Designer UI/UX	Protótipo das telas	15h	R\$ 50,00	R\$ 750,00
Desenvolvedor Front-end	Desenvolvimento das telas	35h	R\$ 55,00	R\$ 1.925,00
Desenvolvedor Back-end	Desenvolvimento da lógica do sistema	40h	R\$ 65,00	R\$ 2.600,00
Administrador de Banco de Dados	Modelagem e criação do banco	15h	R\$ 60,00	R\$ 900,00
Tester/QA	Testes e correções	15h	R\$ 45,00	R\$ 675,00
Gerente de Projeto	Organização e acompanhamento	10h	R\$ 70,00	R\$ 700,00
TOTAL ESTIMADO				R\$ 8.750,00

Fonte: elaborado pelos autores.

9 IMPLEMENTAÇÃO REALIZADA

Este capítulo descreve a implementação concreta do sistema GymControl, do código-fonte ao deploy. Todo o conteúdo dos capítulos 1 a 8 — descritivo computacional, requisitos, modelagem UML, tecnologias previstas, cronograma e orçamento — foi cumprido. Onde a implementação foi além do planejado, isso é registrado nas seções a seguir.

9.1 Visão geral

O sistema foi entregue como uma aplicação web cliente-servidor. O front-end é uma SPA (single-page application) leve em HTML, CSS (Tailwind via CDN) e JavaScript modular. O back-end é uma API REST em Node.js com Express, persistindo dados em MySQL 8. O código está publicado no GitHub e o front-end já está no ar na Vercel.

9.2 Arquitetura

O repositório segue uma estrutura clara que separa responsabilidades: o diretório `lib/` concentra módulos puros (validação de CPF, autenticação JWT, sanitização de logs, migração inicial); `server.js` é o ponto de entrada Express com todas as rotas `/api/*`; `public/` contém o front-end estático com `index.html`, `styles.css` e onze módulos JavaScript em `public/js/` (`core`, `dashboard`, `alunos`, `professores`, `planos`, `pagamentos`, `inadimplentes`, `frequencia`, `backup`, `professor`, `aluno`); `sql/` traz o schema, o seed e consultas de referência; `tests/` guarda os testes automatizados; `docs/` contém a documentação completa; `infra/terraform/` define a infraestrutura como código para AWS.

9.3 Backend

O back-end usa Node.js 20 com módulos ES (ESM, `type=module` no `package.json`). As dependências de produção são: `express` para o roteamento HTTP; `mysql2/promise` como driver MySQL com connection pool; `bcrypt` para hashing de senhas; `jsonwebtoken` para emissão e verificação de JWT; `cookie-parser` para leitura do cookie de sessão; `cors` com `allowlist` via variável de ambiente; `helmet` para definir headers de segurança; `express-rate-limit` para limitar tentativas de login e chamadas à API. Todas as queries usam binds parametrizados, eliminando a possibilidade de SQL injection.

9.4 Frontend

O front-end é estático (sem build step) e funciona em qualquer servidor que sirva arquivos. A estilização usa Tailwind CSS via CDN. O JavaScript foi modularizado em onze arquivos por domínio, carregados em ordem por index.html. Todo o tratamento de eventos é feito por delegação centralizada (atributo data-action), sem nenhum handler inline no HTML. Essa decisão foi necessária para suportar uma Content Security Policy estrita, descrita na seção 9.6.

9.5 Banco de dados

O banco escolhido foi MySQL 8, conforme planejado no QUADRO 11. O schema completo está em sql/01_schema.sql e implementa as oito classes do QUADRO 8 como oito tabelas (usuarios, alunos, professores, planos, pagamentos, treinos, exercicios, frequencias). Todas as chaves estrangeiras estão definidas com ações ON DELETE apropriadas (CASCADE para dependências, SET NULL para vínculos opcionais, RESTRICT onde a integridade exige). ENUMs do MySQL restringem campos categóricos (status, método de pagamento, nível de treino, tipo de usuário). Toda chave estrangeira possui índice. Dados de demonstração estão em sql/02_seed.sql. Em ambiente de desenvolvimento o banco sobe via docker-compose.yml com auto-carga do schema e do seed na primeira inicialização do volume.

9.6 Endurecimento de segurança (RNF02 e RNF07)

Os requisitos não funcionais de controle de acesso (RNF02) e proteção de informações (RNF07) foram atendidos com múltiplas camadas de defesa: senhas armazenadas como hash bcrypt (custo 10); autenticação via JWT em cookie httpOnly com prefixo __Secure-, SameSite=Strict e secure=true; middleware de RBAC aplicado rota a rota conforme o QUADRO 3 (Admin, Professor, Aluno); helmet configurando Content Security Policy estrita (default-src 'self', sem 'unsafe-inline' em script-src), HSTS, X-Frame-Options DENY, X-Content-Type-Options nosniff; express-rate-limit com 120 requisições por minuto na API e 5 por minuto no endpoint de login; validação de CPF com cálculo de dígito verificador; sanitizador de logs que remove CPF, e-mail, hash bcrypt e tokens antes de qualquer console.error; log de auditoria de tentativas de login (sucesso e falha) com e-mail mascarado; redirecionamento HTTPS em produção; JWT_SECRET obrigatório em produção (a aplicação recusa subir com o valor padrão de desenvolvimento).

9.7 Testes automatizados

O sistema possui três camadas de testes automatizados que rodam na integração contínua e localmente. A camada SQL (`sql/04_tests.sql`) verifica que o schema rejeita inserções inválidas (UNIQUE em CPF e CREF, ENUMs, FKs órfãs, CHECK em valor negativo, CASCADE de exercícios). A camada Jest soma 106 testes em seis suítes: `validators`, `log-sanitizer` e `auth` são testados unitariamente sem banco; `api.test.js` e `api.extra.test.js` usam Supertest para exercitar todos os endpoints contra um MySQL real, validando a matriz RBAC do QUADRO 3; `migrate.test.js` cobre o parser SQL do bootstrap. A camada Playwright tem 16 testes em sete arquivos de spec, dirigindo um Chromium real para validar o fluxo completo de login, CRUD de alunos com máscara de CPF, criação de treino com exercícios pelo professor, registro de pagamento, registro de check-in, RBAC visual (o aluno não vê o menu de inadimplentes) e logout. Há também um modo "human-paced" que roda a suíte com pausa de um segundo entre cada ação e grava vídeo, gerando um arquivo MP4 único para revisão humana. A cobertura de linhas do back-end é de 91%, com lib/ a 100%.

9.8 Containerização

A aplicação foi empacotada em uma imagem Docker multi-stage. O primeiro estágio instala apenas as dependências de produção (com compilação nativa do bcrypt). O segundo estágio copia `node_modules` e o código sob um usuário não-root, define HEALTHCHECK apontando para `/healthz` e usa tini como PID 1 para roteamento correto de sinais. O ambiente de desenvolvimento usa `docker-compose.yml` com MySQL 8.4 já configurado para UTF-8 e auto-carga do schema.

9.9 Integração contínua e entrega contínua (CI/CD)

Há três workflows do GitHub Actions: o CI (`.github/workflows/ci.yml`) roda em todo push e pull request executando, nesta ordem, `npm audit`, `lint`, testes SQL, testes Jest, testes Playwright e Trivy (scanner de vulnerabilidades de container). Falha em qualquer etapa bloqueia o merge. O CodeQL (`.github/workflows/codeql.yml`) roda análise estática de segurança semanalmente e em pull requests, usando o conjunto de regras `security-and-quality`. O CD (`.github/workflows/cd.yml`) está implementado mas dormente: só dispara se a variável `ENABLE_CD` do repositório for `true` e o segredo `AWS_DEPLOY_ROLE_ARN` estiver configurado. Ele assume um role da AWS via OIDC (sem credenciais estáticas),

constrói a imagem, faz scan Trivy de novo, faz push para ECR e dispara um deploy no App Runner.

9.10 Deploy

O front-end está hospedado na Vercel em <https://gym-control-pearl.vercel.app> — deploy automático a cada push para main. A infraestrutura AWS necessária para o back-end está totalmente escrita como código em `infra/terraform/`: VPC com duas sub-redes privadas em zonas distintas (sem IGW, sem NAT, sem custo extra de tráfego); RDS MySQL 8 t4g.micro (free-tier por 12 meses), criptografado, em sub-rede privada; ECR para a imagem; App Runner com VPC connector para falar com o RDS, capacidade fixada em uma única instância com concorrência máxima de 25 requisições (controle de custo); Secrets Manager guardando a senha do banco e o JWT_SECRET (ambos gerados aleatoriamente pelo Terraform); CloudWatch Logs com retenção de 7 dias; AWS Budgets opcional com alertas por e-mail. O deploy real não foi efetivado para esta entrega (custo estimado de \$20 USD nos dois meses do projeto), mas todo o ferramental está pronto: bastaria executar `terraform apply` seguido do script `scripts/aws-deploy.sh`.

9.11 Rastreabilidade de requisitos

Esta seção mapeia cada requisito do capítulo 4 ao artefato concreto que o implementa, demonstrando que todos foram efetivamente entregues. Os requisitos funcionais (RF) são listados primeiro, seguidos dos requisitos não funcionais (RNF). Caminhos referem-se à raiz do repositório.

Requisitos funcionais

RF01 — Cadastro de alunos: rota POST `/api/alunos` em `server.js`, restrita ao perfil Admin pelo middleware `requireRole`; tabela alunos em `sql/01_schema.sql` com UNIQUE em CPF e validação de dígito verificador em `lib/validators.js`. Tela: `public/index.html` → seção `#secAlunos`, módulo `public/js/alunos.js`. Testes: `tests/api.test.js` (criação válida, CPF inválido, CPF duplicado) e `tests/e2e/alunos.spec.js` (CRUD com máscara de CPF).

RF02 — Edição e exclusão de alunos: rotas PUT `/api/alunos/:id` e DELETE `/api/alunos/:id` em `server.js`, RBAC Admin; FK de pagamentos e frequencias com ON DELETE CASCADE garante consistência. Tela e ações no mesmo módulo `public/js/alunos.js`. Testes em `tests/api.test.js` e `tests/e2e/alunos.spec.js`.

RF03 — Cadastro de professores: rota POST `/api/professores` em `server.js`, RBAC Admin; tabela `professores` com UNIQUE em CREF; módulo `public/js/professores.js`. Testes: `tests/api.test.js` e `tests/e2e/professores.spec.js`.

RF04 — Cadastro de planos: rotas `/api/planos` em `server.js`, RBAC Admin; tabela `planos` em `sql/01_schema.sql`; módulo `public/js/planos.js`. Testes: `tests/api.test.js`.

RF05 — Registrar pagamentos: rota POST `/api/pagamentos` em `server.js`, RBAC Admin; tabela `pagamentos` com FK para `alunos` e `planos`, ENUM (metodo, status) e CHECK valor > 0; módulo `public/js/pagamentos.js`. Testes: `tests/api.test.js` e `tests/e2e/pagamentos.spec.js`.

RF06 — Consultar inadimplentes: rota GET `/api/inadimplentes` em `server.js` (query com LEFT JOIN entre `alunos` e `pagamentos` do mês corrente), RBAC Admin; módulo `public/js/inadimplentes.js`. Teste visual de RBAC em `tests/e2e/rbac.spec.js` (aluno não vê o menu).

RF07 — Cadastro de treinos: rotas `/api/treinos` e `/api/treinos/:id/exercicios` em `server.js`, RBAC Professor (e Admin); tabelas `treinos` e `exercicios` em `sql/01_schema.sql` com FK CASCADE; módulo `public/js/professor.js`. Teste E2E em `tests/e2e/professor.spec.js` (criação de treino com exercícios).

RF08 — Vincular treinos a alunos: coluna `aluno_id` na tabela `treinos` com FK; tela do professor seleciona o aluno antes de criar o treino; endpoint GET `/api/alunos/me/treinos` serve o painel do aluno (`public/js/aluno.js`). Testes em `tests/api.test.js`.

RF09 — Registrar frequência: rota POST `/api/frequencias` em `server.js`; tabela `frequencias` com FK para `alunos` e UNIQUE (`aluno_id`, `data`); módulo `public/js/frequencia.js`. Teste E2E em `tests/e2e/frequencia.spec.js` (registro de check-in).

RF10 — Relatórios: dashboard em `public/js/dashboard.js` consome GET `/api/dashboard/metrics`, que retorna totais de `alunos ativos`, `pagamentos do mês`, `inadimplentes` e `check-ins do dia` (consultas agregadas em SQL). RBAC Admin. Testes em `tests/api.test.js`.

Requisitos não funcionais

RNF01 — Interface simples e intuitiva: SPA com menu lateral fixo (`public/index.html`), Tailwind CSS aplicando paleta consistente em `public/styles.css`, navegação por data-action sem recarregar a página. Validações em tempo real (máscara de CPF, formato de e-mail) reduzem erros de digitação.

RNF02 — Controle de acesso por tipo de usuário: middleware `requireAuth` + `requireRole` em `lib/auth.js` aplicado a cada rota `/api/*` conforme matriz do QUADRO 3 (Admin, Professor, Aluno); três perfis no schema (tabela `usuarios.tipo`); front-end esconde menus por perfil em

public/js/core.js. Testes: 12 cenários em tests/api.test.js + tests/api.extra.test.js cobrem cada combinação papel × rota; tests/e2e/rbac.spec.js valida visibilidade do menu.

RNF03 — Banco relacional: MySQL 8, schema normalizado em 8 tabelas (sql/01_schema.sql) com FKs, ENUMs, UNIQUE, CHECK e índices em todas as FKs.

Conformidade ACID herdada do MySQL (InnoDB).

RNF04 — Acessível por navegador: aplicação web servida por Express em server.js; front-end estático compatível com qualquer navegador moderno; nenhuma instalação cliente necessária. Front-end já publicado em <https://gym-control-pearl.vercel.app>.

RNF05 — Boa organização visual: hierarquia clara (header com identificação do perfil, menu lateral por domínio, área principal com tabelas paginadas e formulários alinhados); espaçamento e tipografia consistentes via classes utilitárias do Tailwind; ícones discretos guiando ações primárias.

RNF06 — Consultas rápidas: connection pool do mysql2 (10 conexões) em server.js; índices em todas as FKs e em campos de busca (CPF, e-mail, data de pagamento); paginação no front-end. As consultas do dashboard são todas $O(1)$ em número de tabelas tocadas, com agregação feita no SQL e não em JavaScript.

RNF07 — Proteger informações dos usuários: senhas em bcrypt (custo 10) — nunca em texto plano; sessão por JWT em cookie httpOnly + __Secure- + SameSite=Strict (sem acesso por JavaScript do cliente); todas as queries parametrizadas (sem SQL injection); helmet com CSP estrita, HSTS, X-Frame-Options DENY; rate-limit em /api/auth/login (5/min) defendendo contra força bruta; sanitizador de logs em lib/log-sanitizer.js remove CPF, e-mail, hash e token antes de imprimir; JWT_SECRET obrigatório em produção. Detalhamento completo em docs/entrega-final/added/seguranca.md.

10 ACESSO E EXECUÇÃO DO SOFTWARE

10.1 Frontend no ar

O front-end pode ser acessado em <https://gym-control-pearl.vercel.app>. Como o back-end não foi efetivamente publicado, as chamadas /api/* retornam erro de rede; o front-end ainda assim demonstra o layout, a tela de login e as telas dos três perfis quando inspecionado no navegador.

10.2 Repositório de código

O código-fonte completo está em <https://github.com/DyeAllPies/GymControl>. O fork foi feito a partir do repositório original do grupo (<https://github.com/eduardo2580/GymControl>) e as

contribuições foram enviadas de volta via pull request (<https://github.com/eduardo2580/GymControl/pull/1>) para registro.

10.3 Execução local com Docker

Para executar o sistema localmente bastam quatro comandos a partir do diretório do repositório: "git clone <https://github.com/DyeAllPies/GymControl.git> && cd GymControl", "npm install", "docker compose up -d" (sobe o MySQL com schema e seed automaticamente) e "npm start" (sobe a aplicação em <http://localhost:3000>). As contas de demonstração estão no README. O passo a passo completo, incluindo execução sem Docker, está em docs/tutorials/ no repositório.

10.4 Documentação adicional

Além deste relatório, o repositório traz documentação granular em docs/. A pasta docs/entrega-parcial/ guarda este documento na sua versão original (planejamento). A pasta docs/entrega-final/ contém documentação viva: cada um dos dez requisitos funcionais e cada um dos sete requisitos não funcionais tem um arquivo Markdown próprio com referência ao código que o implementa; há também arquivos para cada caso de uso (QUADROS 4, 5 e 6), cada classe do modelo (QUADRO 8), a relação atualizada de tecnologias e descrições das funcionalidades adicionadas além do planejamento original (autenticação, endurecimento de segurança, testes, CI/CD, containerização, backup/restore, preparação do deploy AWS). O guia operacional de deploy está em docs/entrega-final/aws-deploy-guide.md.

11 CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema GymControl busca demonstrar a importância da modelagem e do planejamento no processo de criação de software. Por meio dos elementos da UML, conforme proposto por Booch, Rumbaugh e Jacobson (2006), é possível visualizar melhor a estrutura, o comportamento e os componentes do sistema antes da implementação, reduzindo riscos e retrabalho na fase de desenvolvimento.

Retomando a hipótese formulada na introdução, o planejamento apresentado neste trabalho indica que um sistema web simples, desenvolvido com tecnologias amplamente difundidas, é viável e adequado para atender às necessidades de academias de pequeno e médio porte. As funcionalidades essenciais foram identificadas, modeladas e descritas,

permitindo maior organização no controle de alunos, professores, planos, pagamentos, treinos e frequência.

Dessa forma, o projeto contribui para a compreensão prática dos conceitos de engenharia de software, modelagem de sistemas e desenvolvimento de aplicações, conforme abordados por Pressman (2016) e Sommerville (2019). Para a próxima etapa, será realizada a implementação do sistema com base no planejamento apresentado, utilizando as tecnologias escolhidas e seguindo o cronograma definido.

REFERÊNCIAS

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML: guia do usuário**. 2. ed. Rio de Janeiro: Elsevier, 2006. 474 p.

PRESSMAN, Roger S. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: McGraw-Hill, 2016. 968 p.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019. 744 p.